Active Record Validations

This guide teaches you how to validate the state of objects before they go into the database using Active Record's validations feature.

1 Validations Overview

Here's an example of a very simple validation:

```
class Person < ActiveRecord::Base
  validates :name, presence: true
end

Person.create(name: "John Doe").valid? # => true
Person.create(name: nil).valid? # => false
```

As you can see, our validation lets us know that our `Person` is not valid without a `name` attribute. The second `Person` will not be persisted to the database.

Before we dig into more details, let's talk about how validations fit into the big picture of your application.

1.1 Why Use Validations?

Validations are used to ensure that only valid data is saved into your database. For example, it may be important to your application to ensure that every user provides a valid email address and mailing address. Model-level validations are the best way to ensure that only valid data is saved into your database. They are database agnostic, cannot be bypassed by end users, and are convenient to test and maintain. Rails makes them easy to use, provides built-in helpers for common needs, and allows you to create your own validation methods as well.

There are several other ways to validate data before it is saved into your database, including native database constraints, client-side validations, controller-level validations. Here's a summary of the pros and cons:

- Database constraints and/or stored procedures make the validation mechanisms database-dependent and can make testing and maintenance more difficult. However, if your database is used by other applications, it may be a good idea to use some constraints at the database level. Additionally, database-level validations can safely handle some things (such as uniqueness in heavily-used tables) that can be difficult to implement otherwise.
- Client-side validations can be useful, but are generally unreliable if used alone. If they are implemented using JavaScript, they may be bypassed if JavaScript is turned off in the user's browser. However, if combined with other techniques, client-side validation can be a convenient way to provide users with immediate feedback as they use your site.
- Controller-level validations can be tempting to use, but often become unwieldy and difficult to test and maintain. Whenever possible, it's a good idea to keep your controllers skinny, as it will make your application a pleasure to work with in the long run.

Choose these in certain, specific cases. It's the opinion of the Rails team that model-level validations are the most appropriate in most circumstances.